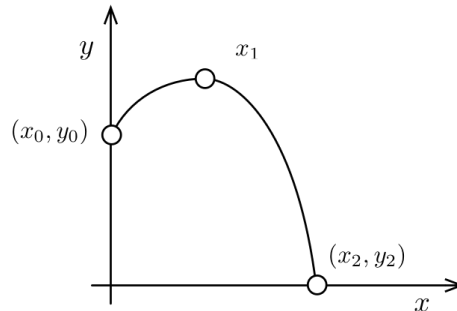


Problemi di Progettazione di Curve

Problemi di Progettazione di Curve

Si vuole progettare una arcata a ridosso di una parete verticale



La curva che descrive l'arcata:

- Deve essere *ancorata ad un punto noto* (x_0, y_0) sulla parete
- Deve essere *ancorata ad un punto noto* (x_2, y_2) a terra
- Deve *raggiungere l'altezza massima* per $x = x_1$ (con x_1 noto)

Problemi di Progettazione di Curve

Un approccio: trattiamo la curva come una funzione $f(x)$

In questo modo possiamo *tradurre le condizioni in equazioni*:

- Deve essere ancorata ad un punto noto (x_0, y_0) sulla parete

$$f(x_0) = y_0$$

- Deve essere ancorata ad un punto noto (x_2, y_2) a terra

$$f(x_2) = y_2$$

- Deve raggiungere l'altezza massima per $x = x_1$ (con x_1 noto)

$$f'(x_1) = y_1$$

Così come sono ci dicono ben poco...

Problemi di Progettazione di Curve

Ci serve una assunzione sulla classe della funzione $f(x)$

Per esempio possiamo assumere che $f(x)$ sia polinomiale, i.e.:

$$f(x) = \sum_{i=0}^n \alpha_i x^i$$

Le nostre condizioni allora diventano:

$$\text{passaggio per } (x_0, y_0) \quad \sum_{i=0}^n \alpha_i x_0^i = y_0 \quad (1)$$

$$\text{passaggio per } (x_1, y_1) \quad \sum_{i=0}^n \alpha_i x_1^i = y_1 \quad (2)$$

$$\text{annullamento di } f'(x_1) \quad \sum_{i=1}^n i \alpha_i x_1^{i-1} = 0 \quad (3)$$

Problemi di Progettazione di Curve

Ci siamo quasi! Guardiamole meglio:

$$\text{passaggio per } (x_0, y_0) \quad \sum_{i=0}^n \alpha_i x_0^i = y_0 \quad (4)$$

$$\text{passaggio per } (x_1, y_1) \quad \sum_{i=0}^n \alpha_i x_1^i = y_1 \quad (5)$$

$$\text{annullamento di } f'(x_1) \quad \sum_{i=1}^n i \alpha_i x_1^{i-1} = 0 \quad (6)$$

- Le incognite sono i *parametri della funzione* α_i e non le x !
- Abbiamo tre condizioni, quindi ci servono tre variabili
 - i.e. $\alpha_0, \alpha_1, \alpha_2 \Rightarrow$ secondo grado

Problemi di Progettazione di Curve

In questo modo otteniamo il sistema di equazioni:

$$\alpha_2 x_0^2 + \alpha_1 x_0 + \alpha_0 = y_0 \quad (7)$$

$$\alpha_2 x_1^2 + \alpha_1 x_1 + \alpha_0 = y_1 \quad (8)$$

$$2\alpha_2 x_1 + \alpha_1 = 0 \quad (9)$$

- Che è *lineare* nelle incognite $\alpha_2, \alpha_1, \alpha_0$

La tecnica vista è un metodo generale per progettare curve:

- Si ipotizza una struttura per la curva da costruire (e.g. polinomio)

- Si determina il numero di gradi di libertà (coefficienti) necessari
- Si traducono i vincoli del problema in equazioni
- Si risolvono le equazioni per determinare i parametri

Soluzione di Sistemi Lineari

Come possiamo risolvere il nostro sistema?

$$\alpha_2 x_0^2 + \alpha_1 x_0 + \alpha_0 = y_0 \quad (10)$$

$$\alpha_2 x_2^2 + \alpha_1 x_2 + \alpha_0 = y_2 \quad (11)$$

$$2\alpha_2 x_1 + \alpha_1 = 0 \quad (12)$$

Ci sono una varietà di metodi disponibili:

- E.g. [eliminazione di Gauss](#)
- E.g. [Singular Value Decomposition](#)

Possiamo pensare di eseguirli a mano

- ...Oppure possiamo usare l'*implementazione fornita da* `numpy`

Soluzione di Sistemi Lineari

Il primo passo è formulare il sistema in *forma matriciale*

$$\alpha_2 x_0^2 + \alpha_1 x_0 + \alpha_0 = y_0 \quad (13)$$

$$\alpha_2 x_2^2 + \alpha_1 x_2 + \alpha_0 = y_2 \quad (14)$$

$$2\alpha_2 x_1 + \alpha_1 = 0 \quad (15)$$

Nel nostro caso otteniamo:

$$\begin{pmatrix} x_0^2 & x_0 & 1 \\ x_2^2 & x_2 & 1 \\ 2x_1 & 1 & 0 \end{pmatrix} \begin{pmatrix} \alpha_2 \\ \alpha_1 \\ \alpha_0 \end{pmatrix} = \begin{pmatrix} y_0 \\ y_2 \\ 0 \end{pmatrix}$$

Soluzione di Sistemi Lineari

Quindi costruiamo degli array corrispondenti a:

- La matrice dei coefficienti (di solito denotata con A)
- La colonna dei termini noti (di solito denotata con b)

Di nuovo, nel nostro caso abbiamo:

$$A = \begin{pmatrix} x_0^2 & x_0 & 1 \\ x_2^2 & x_2 & 1 \\ 2x_1 & 1 & 0 \end{pmatrix} \begin{pmatrix} \alpha_2 \\ \alpha_1 \\ \alpha_0 \end{pmatrix} \quad b = \begin{pmatrix} y_0 \\ y_2 \\ 0 \end{pmatrix}$$

Soluzione di Sistemi Lineari

Per esempio, possiamo usare:

```
In [2]: import numpy as np

x0, x1, x2 = 0, 2, 6
y0, y2 = 3, 0

A = np.array([[x0**2, x0, 1],
               [x2**2, x2, 1],
               [2*x1, 1, 0]])

b = np.array([y0, y2, 0])
print('A = ')
print(A)
print('b = ')
print(b)
```

```
A =
[[ 0  0  1]
 [36  6  1]
 [ 4  1  0]]
b =
[3 0 0]
```

Soluzione di Sistemi Lineari

Per risolvere il sistema possiamo usare la funzione `numpy.linalg.solve`

```
In [3]: help(np.linalg.solve)
```

Help on `_ArrayFunctionDispatcher` in module `numpy.linalg`:

`solve(a, b)`

Solve a linear matrix equation, or system of linear scalar equations.

Computes the "exact" solution, ``x``, of the well-determined, i.e., full rank, linear matrix equation ``ax = b``.

Parameters

`a` : `(..., M, M)` array_like

Coefficient matrix.

`b` : `{(M,), (..., M, K)}`, array_like

Ordinate or "dependent variable" values.

Returns

`x` : `{(..., M,), (... , M, K)}` ndarray

Solution to the system `a x = b`. Returned shape is `(..., M)` if `b` is shape `(M,)` and `(..., M, K)` if `b` is `(..., M, K)`, where the `"..."` part

is

broadcasted between `a` and `b`.

Raises

`LinAlgError`

If ``a`` is singular or not square.

See Also

`scipy.linalg.solve` : Similar function in SciPy.

Notes

Broadcasting rules apply, see the ``numpy.linalg`` documentation for details.

The solutions are computed using LAPACK routine ``_gesv``.

``a`` must be square and of full-rank, i.e., all rows (or, equivalently, columns) must be linearly independent; if either is not true, use ``lstsq`` for the least-squares best "solution" of the system/equation.

.. versionchanged:: 2.0

The `b` array is only treated as a shape `(M,)` column vector if it is exactly 1-dimensional. In all other instances it is treated as a stack

k

of `(M, K)` matrices. Previously `b` would be treated as a stack of `(M,)` vectors if `b.ndim` was equal to `a.ndim - 1`.

References

.. [1] G. Strang, **Linear Algebra and Its Applications**, 2nd Ed., Orland

o,

Examples

Solve the system of equations:

$x_0 + 2 * x_1 = 1$ and
 $3 * x_0 + 5 * x_1 = 2$:

```
>>> import numpy as np
>>> a = np.array([[1, 2], [3, 5]])
>>> b = np.array([1, 2])
>>> x = np.linalg.solve(a, b)
>>> x
array([-1.,  1.])
```

Check that the solution is correct:

```
>>> np.allclose(np.dot(a, x), b)
True
```

Soluzione di Sistemi Lineari

Vediamo un esempio di soluzione completa

```
In [4]: import numpy as np

x0, x1, x2 = 0, 2, 6
y0, y2 = 3, 0

A = np.array([[x0**2, x0, 1],
               [x2**2, x2, 1],
               [2*x1, 1, 0]])

b = np.array([y0, y2, 0])

alpha = np.linalg.solve(A, b)
print(f'alpha = {alpha}')
alpha_2, alpha_1, alpha_0 = alpha

alpha = [-0.25  1.    3. ]
```

Disegnare la Curva

Possiamo poi procedere a disegnare la curva

Iniziamo valutando la nostra curva per un insieme di valori x

```
In [5]: x = np.linspace(x0, x2, 10)
y = alpha_2 * x**2 + alpha_1 * x + alpha_0
print(f'x = {x}')
print(f'y = {y}')
```

```
x = [0.          0.66666667 1.33333333 2.          2.66666667 3.33333333
      4.          4.66666667 5.33333333 6.          ]
y = [3.          3.55555556 3.88888889 4.          3.88888889 3.55555556
      3.          2.22222222 1.22222222 0.          ]
```

- La funzione `numpy.linspace` genera un array di valori equispaziati
 - ...Esattamente come la `linspace` che avevamo definito tempo fa
- Per calcolare `y` facciamo uso del calcolo vettoriale
 - E.g. `x**2` è un array con i quadrati degli elementi in `x`

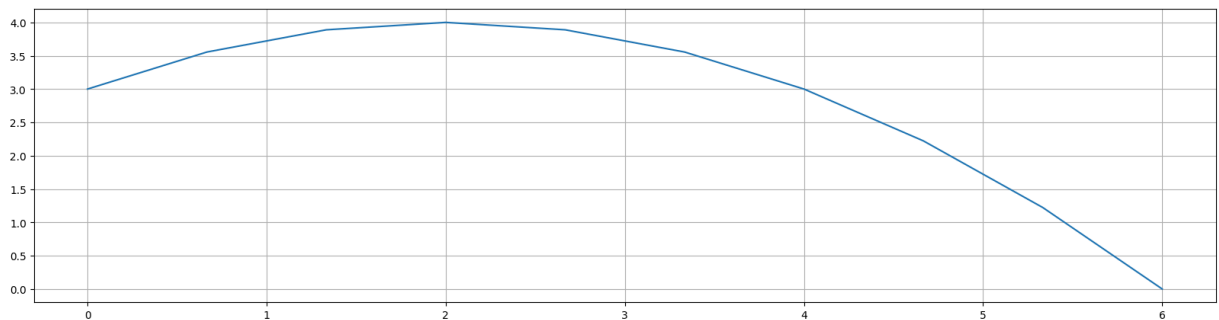
Disegnare la Curva

Possiamo poi procedere a disegnare la curva

Il grafico vero e proprio possiamo farlo nel solito modo

In [6]: `from matplotlib import pyplot as plt`

```
plt.figure(figsize=(20, 5))
plt.plot(x, y)
plt.grid()
plt.show()
```



Disegnare la Curva

Includiamo il codice nel pacchetto `base.arc`

- Una funzione `base.arc.fit_curve` per ottenere i coefficienti
- Una funzione `base.arc.draw_curve` per disegnare la curva

```
In [7]: %load_ext autoreload
        %autoreload 2
        from base import arc
        x0, x1, x2 = 0, 2, 6
        y0, y2 = 3, 0
        alpha_2, alpha_1, alpha_0 = arc.fit_curve(x0, x1, x2, y0, y2)
        arc.draw_curve(alpha_2, alpha_1, alpha_0, x0, x2)
```

